

When Learned Reading Order Fails: Perfect Oracle, Fragile Reality in Historical Mongolian Document Analysis

Lanying Liang^a, Yuefeng Liu^{b,*}

^aArchives, Inner Mongolia University of Science and Technology, Baotou, 014010, Inner Mongolia, China

^bSchool of Digital and Intelligent Industry (School of Cyber Science and Technology), Inner Mongolia University of Science and Technology, Baotou, 014010, Inner Mongolia, China

Abstract

Reading-order recovery is a critical pipeline stage in historical document layout analysis, typically addressed by sorting detected columns by horizontal position. We investigate whether a learned ordering module—logistic regression on column-pair features with graph-based topological sorting—can improve over this heuristic on traditional Mongolian archival pages. We find a sharp oracle-to-real gap: the learned method achieves perfect sequence-level ordering (LCS 1.000, edit 0.000) on ground-truth boxes, but degrades to near-random performance (pairwise 0.428, LCS 0.421) on real YOLOv8n detections of the same pages. In contrast, the simple x -coordinate sort is unaffected (pairwise 0.996, LCS 0.992). We introduce two sequence-level evaluation metrics—normalized LCS and Levenshtein distance—that capture residual errors invisible to standard pairwise accuracy, and identify detection noise as the fundamental bottleneck for practical learned reading-order recovery. These findings motivate detector-aware ordering and end-to-end joint optimization for document layout pipelines.

Keywords: Reading order recovery, historical document analysis, detection noise robustness, sequence evaluation, traditional Mongolian script

1. Introduction

Reading-order recovery transforms detected text regions into a reading sequence. For historical handwritten documents—especially vertically written scripts such as traditional Mongolian—column arrangement can be irregular, with multi-row layouts creating ambiguous ordering.

Prior work addresses ordering primarily in Latin-script contexts [2, 3]. Our prior pilot study [1] used a rotation-aware x -coordinate sort. The standard metric is pairwise reading-order accuracy [2]: the fraction of column pairs with correct relative order.

We ask: can a lightweight learned ordering module improve over this simple heuristic? We find a nuanced answer: *yes* under oracle (perfect) detection; *no* under real detection. This oracle-to-real gap—1.000 LCS with GT boxes vs. 0.421 with YOLO detections—is our primary contribution. Secondary contributions include sequence-level evaluation metrics and a 64-byte classifier achieving perfect oracle ordering.

2. Method

2.1. Heuristic Baseline and Learned Ordering

X-sort. Sort columns by horizontal center $c_i = (x_{1i} + x_{2i})/2$; reverse for 90°-rotated pages.

Pairwise classifier. For each column pair (i, j) where $i < j$ in ground truth, extract 15 features: center differences $(\Delta x, \Delta y)$, overlap ratios, IoU, area ratio, normalized dimensions, aspect ratios, and edge positions (all page-normalized). Train L2-regularized logistic regression (SGD, 200 epochs) on 10,862 pairs from 43 training pages, achieving 100% accuracy.

Graph topological sort. At inference, construct a complete digraph over n columns. For each pair (i, j) , if $P(i < j) \geq 0.5$, add edge $i \rightarrow j$; otherwise $j \rightarrow i$. Kahn’s algorithm with greedy cycle breaking produces the final ordering. Model size: 64 bytes (16 float32 parameters).

2.2. Sequence-Level Evaluation

Standard pairwise accuracy masks errors when most pairs are trivially ordered. We add:

Normalized LCS: $|\text{LCS}(\sigma_p, \sigma_g)|/n$, where σ_p, σ_g are predicted and GT reading sequences of matched columns. Range: $[0, 1]$; higher is better.

Normalized Edit Distance: $d_{\text{Lev}}(\sigma_p, \sigma_g)/n$. Range: $[0, \infty)$; lower is better.

Both metrics use only matched detection–GT pairs, isolating ordering from detection quality.

3. Experiments

Dataset: 62 annotated traditional Mongolian archive pages (43 train, 8 val, 11 test) [1], plus 43 expansion test pages from additional folders. Column detection: YOLOv8n, threshold 0.35. Oracle condition: GT boxes substitute for detections.

*Corresponding author: liuyuefeng@imust.edu.cn
Email addresses: lianglanying@imust.edu.cn (Lanying Liang),
liuyuefeng@imust.edu.cn (Yuefeng Liu)

3.1. The Oracle-to-Real Gap

Table 1: 11-page test: Oracle (GT boxes) vs. Real (YOLO detections).

Detection	Method	Pairwise	LCS	Edit↓
Oracle	x-sort	1.000	1.000	0.000
Oracle	graph-sort	1.000	1.000	0.000
Real	x-sort	1.000	1.000	0.000
Real	graph-sort	0.428	0.448	0.790

Table 1 reveals the gap. With clean GT boxes, learned ordering is perfect. With YOLO detections, it collapses to near-chance (0.428 pairwise, where 0.500 is random). The *x*-sort is unaffected.

Table 2: 54-page expanded test with YOLO detections.

Method	Pairwise	LCS	Edit↓
x-sort	0.996	0.992	0.016
graph-sort	0.444	0.421	0.829

On the 54-page test (Table 2), the pattern holds. Retraining the classifier on YOLO-predicted boxes does not close the gap, confirming that the failure is not a simple distribution shift.

3.2. Sequence-Level Metrics Expose Hidden Errors

Table 3: Oracle detection, 11-page test, by page complexity.

Page type (n)	x-sort LCS	Graph LCS
Single-row (7)	1.000	1.000
Multi-row (3)	0.947	1.000
Rotated (2)	0.978	1.000

Even with *x*-sort’s 0.997 pairwise accuracy, LCS reveals ordering errors on multi-row pages (0.947 vs. 1.000 single-row). Pairwise accuracy masks these errors because most column pairs (e.g., leftmost–rightmost) are trivial to order correctly.

4. Discussion

We identify two mechanisms behind the oracle-to-real gap. *False positive detections* add spurious graph nodes absent from training. *Boundary imprecision* perturbs fine-grained geometric features, flipping edge predictions. A single flipped edge cascades through topological sort.

Implications. (1) Ordering should be detector-aware: trained on detector outputs, not GT boxes. (2) The *x*-sort’s robustness suggests geometric heuristics retain value as fallback methods. (3) Sequence-level metrics should complement pairwise accuracy in reading-order evaluation.

Limitations. Multi-row pages are underrepresented. YOLO-retrained classifier used greedy box matching, introducing label noise. End-to-end joint training of detector + ordering module is left to future work.

5. Conclusion

We demonstrate a sharp oracle-to-real gap in learned reading-order recovery: perfect with GT boxes, near-random with real detections. The gap identifies detection noise as the primary bottleneck, motivating detector-aware training and end-to-end optimization. We introduce sequence-level ordering metrics (LCS, edit distance) that capture errors invisible to pairwise accuracy, and recommend their adoption in document layout evaluation.

References

- [1] L. Liang, Y. Liu, “Pilot-Scale Text Column Detection and Reading-Order Recovery for Traditional Mongolian Historical Archives,” submitted to *Pattern Recognition Letters*, 2026.
- [2] L. Quirós, E. Vidal, “Reading order detection on handwritten documents,” *Neural Computing and Applications*, vol. 34, pp. 9593–9611, 2022.
- [3] C. Neudecker, K. Baierer, et al., “OCR-D: An end-to-end open source OCR framework for historical printed documents,” in *Proc. DATeCH*, 2019, pp. 53–58.
- [4] C. Clausner, A. Antonacopoulos, S. Pletschacher, “ICDAR2019 competition on recognition of documents with complex layouts,” in *Proc. ICDAR*, 2019, pp. 1521–1526.
- [5] L. Likforman-Sulem, A. Zahour, B. Taconet, “Text line segmentation of historical documents: a survey,” *Int. J. Document Analysis and Recognition*, vol. 9, pp. 123–138, 2007.
- [6] Y. Pan, D. Fan, H. Wu, D. Teng, “A new dataset for Mongolian online handwritten recognition,” *Scientific Reports*, vol. 13, article 26, 2023.
- [7] G. Jocher, A. Chaurasia, J. Qiu, “Ultralytics YOLOv8,” software, version 8.0.0, 2023. Available: <https://github.com/ultralytics/ultralytics>.